

# Data Structure Time Complexity Cheat Sheet

Python vs C++ — side-by-side comparison of core operations, average and worst case

■ Green =  $O(1)$  /  $O(\log n)$  — fast ■ Amber =  $O(n)$  — linear ■ Red =  $O(n \log n)$  or worse — slow

## 1. Array / Dynamic List

Python: `list` | C++: `std::vector`

| Operation               | Python method                                  | C++ method                                          | Py avg        | Py worst      | C++ avg       | C++ worst     | Notes                           |
|-------------------------|------------------------------------------------|-----------------------------------------------------|---------------|---------------|---------------|---------------|---------------------------------|
| Index access            | <code>a[i]</code>                              | <code>v[i]</code> / <code>v.at(i)</code>            | $O(1)$        | $O(1)$        | $O(1)$        | $O(1)$        | Direct memory offset            |
| Append end              | <code>a.append(x)</code>                       | <code>v.push_back(x)</code>                         | $O(1)$        | $O(n)$        | $O(1)$        | $O(n)$        | Worst = resize/reallocation     |
| Insert at index         | <code>a.insert(i, x)</code>                    | <code>v.insert(v.begin()+i, x)</code>               | $O(n)$        | $O(n)$        | $O(n)$        | $O(n)$        | Shifts elements after index     |
| Delete at index         | <code>del a[i]</code> / <code>a.pop(i)</code>  | <code>v.erase(v.begin()+i)</code>                   | $O(n)$        | $O(n)$        | $O(n)$        | $O(n)$        | Shifts elements to fill gap     |
| Remove by value         | <code>a.remove(x)</code>                       | <code>v.erase(remove(v.begin(), v.end(), x))</code> | $O(n)$        | $O(n)$        | $O(n)$        | $O(n)$        | Removes first match             |
| Pop end                 | <code>a.pop()</code>                           | <code>v.pop_back()</code>                           | $O(1)$        | $O(1)$        | $O(1)$        | $O(1)$        |                                 |
| Search (unsorted)       | <code>x in a</code>                            | <code>find(v.begin(), v.end(), x)</code>            | $O(n)$        | $O(n)$        | $O(n)$        | $O(n)$        | Linear scan                     |
| Search (sorted, binary) | <code>bisect.bisect_left(a, x)</code>          | <code>lower_bound(v.begin(), v.end(), x)</code>     | $O(\log n)$   | $O(\log n)$   | $O(\log n)$   | $O(\log n)$   |                                 |
| Sort                    | <code>a.sort()</code> / <code>sorted(a)</code> | <code>sort(v.begin(), v.end())</code>               | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | Timsort (Py) vs Introsort (C++) |

## 2. Linked List

Python: `collections.deque` (doubly linked) | C++: `std::list`

| Operation                             | Python method                | C++ method                                                  | Py avg | Py worst | C++ avg | C++ worst | Notes                                           |
|---------------------------------------|------------------------------|-------------------------------------------------------------|--------|----------|---------|-----------|-------------------------------------------------|
| Access front                          | <code>d[0]</code>            | <code>lst.front()</code>                                    | $O(1)$ | $O(1)$   | $O(1)$  | $O(1)$    |                                                 |
| Access back                           | <code>d[-1]</code>           | <code>lst.back()</code>                                     | $O(1)$ | $O(1)$   | $O(1)$  | $O(1)$    |                                                 |
| Access by index                       | <code>d[i]</code>            | <code>advance(it, i)</code>                                 | $O(n)$ | $O(n)$   | $O(n)$  | $O(n)$    | Must traverse from an end                       |
| Insert front                          | <code>d.appendleft(x)</code> | <code>lst.push_front(x)</code>                              | $O(1)$ | $O(1)$   | $O(1)$  | $O(1)$    |                                                 |
| Insert back                           | <code>d.append(x)</code>     | <code>lst.push_back(x)</code>                               | $O(1)$ | $O(1)$   | $O(1)$  | $O(1)$    |                                                 |
| Delete front                          | <code>d.popleft()</code>     | <code>lst.pop_front()</code>                                | $O(1)$ | $O(1)$   | $O(1)$  | $O(1)$    |                                                 |
| Delete back                           | <code>d.pop()</code>         | <code>lst.pop_back()</code>                                 | $O(1)$ | $O(1)$   | $O(1)$  | $O(1)$    |                                                 |
| Insert/delete middle (given iterator) | — (no $O(1)$ equiv)          | <code>lst.insert(it, x)</code> / <code>lst.erase(it)</code> | $O(1)$ | $O(1)$   | $O(1)$  | $O(1)$    | Python deque has no direct node-iterator insert |
| Search                                | <code>x in d</code>          | <code>find(lst.begin(), lst.end(), x)</code>                | $O(n)$ | $O(n)$   | $O(n)$  | $O(n)$    | No random access                                |

## 3. Stack & Queue

Python: `list` (stack) / `collections.deque` (queue) | C++: `std::stack` / `std::queue`

| Operation      | Python (stack)           | Python (queue)           | C++ (stack)            | C++ (queue)            | Complexity |
|----------------|--------------------------|--------------------------|------------------------|------------------------|------------|
| Push / enqueue | <code>s.append(x)</code> | <code>q.append(x)</code> | <code>s.push(x)</code> | <code>q.push(x)</code> | $O(1)$     |
| Pop / dequeue  | <code>s.pop()</code>     | <code>q.popleft()</code> | <code>s.pop()</code>   | <code>q.pop()</code>   | $O(1)$     |
| Peek top/front | <code>s[-1]</code>       | <code>q[0]</code>        | <code>s.top()</code>   | <code>q.front()</code> | $O(1)$     |
| Is empty       | <code>not s</code>       | <code>not q</code>       | <code>s.empty()</code> | <code>q.empty()</code> | $O(1)$     |

Worst case for push:  $O(n)$  amortized to  $O(1)$  — occasional reallocation when the underlying buffer resizes. **Common trap:** don't use Python `list` as a queue — `list.pop(0)` is  $O(n)$  because it shifts every remaining element. Always use `collections.deque` for queue behavior.

4. Hash Map (Dict)

Python: dict | C++: std::unordered\_map

| Operation         | Python method        | C++ method                 | Py avg | Py worst | C++ avg | C++ worst | Notes                                      |
|-------------------|----------------------|----------------------------|--------|----------|---------|-----------|--------------------------------------------|
| Insert / update   | d[k] = v             | m[k] = v / m.insert({k,v}) | O(1)   | O(n)     | O(1)    | O(n)      | Worst = hash collisions                    |
| Lookup / access   | d[k] / d.get(k)      | m[k] / m.at(k)             | O(1)   | O(n)     | O(1)    | O(n)      | m[k] auto-inserts if missing!              |
| Key exists?       | k in d               | m.count(k) / m.find(k)     | O(1)   | O(n)     | O(1)    | O(n)      | Use before m[k] to avoid accidental insert |
| Delete            | del d[k] / d.pop(k)  | m.erase(k)                 | O(1)   | O(n)     | O(1)    | O(n)      |                                            |
| Get w/ default    | d.get(k, default)    | m.find(k)!=m.end()?...     | O(1)   | O(n)     | O(1)    | O(n)      | C++ has no direct one-liner                |
| Iterate all items | for k,v in d.items() | for(auto& [k,v] : m)       | O(n)   | O(n)     | O(n)    | O(n)      |                                            |

5. Set (Hash-based)

Python: set / frozenset | C++: std::unordered\_set

| Operation       | Python method              | C++ method                | Py avg                | Py worst              | C++ avg               | C++ worst             | Notes                                     |
|-----------------|----------------------------|---------------------------|-----------------------|-----------------------|-----------------------|-----------------------|-------------------------------------------|
| Insert          | s.add(x)                   | s.insert(x)               | O(1)                  | O(n)                  | O(1)                  | O(n)                  | Worst = hash collisions                   |
| Membership test | x in s                     | s.count(x) / s.find(x)    | O(1)                  | O(n)                  | O(1)                  | O(n)                  | Main reason to use a set                  |
| Delete          | s.remove(x) / s.discard(x) | s.erase(x)                | O(1)                  | O(n)                  | O(1)                  | O(n)                  | remove raises if missing, discard doesn't |
| Union           | a   b / a.union(b)         | manual loop (no built-in) | O(len(a)+len(b))      | O(len(a)+len(b))      | O(len(a)+len(b))      | O(len(a)+len(b))      | C++ has no set union method               |
| Intersection    | a & b / a.intersection(b)  | manual loop (no built-in) | O(min(len(a),len(b))) | O(min(len(a),len(b))) | O(min(len(a),len(b))) | O(min(len(a),len(b))) |                                           |

"HashSet" / "HashMap" is a general CS concept, not a Java-only term. Java just names its classes literally. Python's set/dict and C++'s unordered\_set/unordered\_map ARE hash tables internally — same concept, different names. frozenset is Python's immutable set (usable as a dict key); C++ has no direct equivalent (closest: a const unordered\_set).

6. Ordered Map & Set (Tree-based, sorted)

Python: sortedcontainers.SortedDict/SortedSet (no built-in) | C++: std::map / std::set

| Operation          | Python method            | C++ method                        | Py avg   | Py worst | C++ avg  | C++ worst | Notes                                       |
|--------------------|--------------------------|-----------------------------------|----------|----------|----------|-----------|---------------------------------------------|
| Insert             | sd[k] = v / ss.add(x)    | m[k]=v / s.insert(x)              | O(log n) | O(log n) | O(log n) | O(log n)  | Red-Black tree in C++                       |
| Lookup             | sd[k] / x in ss          | m[k] / s.count(x)                 | O(log n) | O(log n) | O(log n) | O(log n)  |                                             |
| Delete             | del sd[k] / ss.remove(x) | m.erase(k) / s.erase(x)           | O(log n) | O(log n) | O(log n) | O(log n)  |                                             |
| In-order traversal | for k in sd              | for(auto& [k,v] : m)              | O(n)     | O(n)     | O(n)     | O(n)      | Always sorted order                         |
| Min / Max          | sd.peekitem(0)/(-1)      | m.begin() / m.rbegin()            | O(log n) | O(log n) | O(log n) | O(log n)  |                                             |
| Nearest key ≥/≤ x  | sd.irange(minimum=x)     | m.lower_bound(x) / upper_bound(x) | O(log n) | O(log n) | O(log n) | O(log n)  | The main reason to reach for this structure |

std::map vs std::unordered\_map (the C++ confusion, resolved): they are two genuinely different data structures, not a naming variant

- unordered\_map = hash table → no ordering, O(1) avg — this is "the hashmap"
- map = red-black tree → always sorted by key, O(log n) — use when you need sorted iteration, range queries, or nearest-key lookups

Python side: no built-in sorted map/set in the standard library. Regular dict (3.7+) preserves insertion order only, NOT sorted order — don't confuse the two. collections.OrderedDict also only preserves insertion order (largely redundant since 3.7). For true sorted-key behavior like C++'s std::map, install sortedcontainers (SortedDict / SortedSet) — C-optimized, O(log n), the closest real equivalent.

7. Heap / Priority Queue

Python: `heapq` (min-heap only) | C++: `std::priority_queue` (max-heap by default)

| Operation             | Python method                    | C++ method                                | Py avg                        | Py worst                      | C++ avg                       | C++ worst                     | Notes                                                                          |
|-----------------------|----------------------------------|-------------------------------------------|-------------------------------|-------------------------------|-------------------------------|-------------------------------|--------------------------------------------------------------------------------|
| Push                  | <code>heapq.heappush(h,x)</code> | <code>pq.push(x)</code>                   | <b><math>O(\log n)</math></b> | <b><math>O(\log n)</math></b> | <b><math>O(\log n)</math></b> | <b><math>O(\log n)</math></b> |                                                                                |
| Pop min/max           | <code>heapq.heappop(h)</code>    | <code>pq.pop()</code>                     | <b><math>O(\log n)</math></b> | <b><math>O(\log n)</math></b> | <b><math>O(\log n)</math></b> | <b><math>O(\log n)</math></b> | C++ <code>pop()</code> doesn't return the value — use <code>top()</code> first |
| Peek min/max          | <code>h[0]</code>                | <code>pq.top()</code>                     | <b><math>O(1)</math></b>      | <b><math>O(1)</math></b>      | <b><math>O(1)</math></b>      | <b><math>O(1)</math></b>      |                                                                                |
| Build heap from array | <code>heapq.heapify(a)</code>    | <code>make_heap(v.begin(),v.end())</code> | <b><math>O(n)</math></b>      | <b><math>O(n)</math></b>      | <b><math>O(n)</math></b>      | <b><math>O(n)</math></b>      | Not $O(n \log n)$ — common interview gotcha                                    |

Python's `heapq` is min-heap only — negate values for a max-heap. C++'s `priority_queue` is max-heap by default — pass `greater<int>` for a min-heap.

8. String

Python: `str` (immutable) | C++: `std::string` (mutable)

| Operation             | Python method                       | C++ method                              | Py avg                                                  | Py worst                         | C++ avg                            | C++ worst                        | Notes                                                 |
|-----------------------|-------------------------------------|-----------------------------------------|---------------------------------------------------------|----------------------------------|------------------------------------|----------------------------------|-------------------------------------------------------|
| Index access          | <code>s[i]</code>                   | <code>s[i] / s.at(i)</code>             | <b><math>O(1)</math></b>                                | <b><math>O(1)</math></b>         | <b><math>O(1)</math></b>           | <b><math>O(1)</math></b>         |                                                       |
| Concatenation         | <code>a + b</code>                  | <code>a + b</code>                      | <b><math>O(n)</math></b>                                | <b><math>O(n)</math></b>         | <b><math>O(n)</math></b>           | <b><math>O(n)</math></b>         | Python creates a new string each time (immutable)     |
| Concatenation in loop | <code>''.join(list_of_str)</code>   | <code>s += x</code> (in loop)           | <b><math>O(n^2)</math> if <math>a+=b</math> in loop</b> | <b><math>O(n^2)</math></b>       | <b><math>O(n)</math> amortized</b> | <b><math>O(n^2)^*</math></b>     | Always prefer <code>join()</code> in Python           |
| Substring / slicing   | <code>s[i:j]</code>                 | <code>s.substr(i, len)</code>           | <b><math>O(k)</math></b>                                | <b><math>O(k)</math></b>         | <b><math>O(k)</math></b>           | <b><math>O(k)</math></b>         | $k$ = length of substring                             |
| Search substring      | <code>s.find(sub) / sub in s</code> | <code>s.find(sub)</code>                | <b><math>O(n)</math></b>                                | <b><math>O(n \cdot m)</math></b> | <b><math>O(n)</math></b>           | <b><math>O(n \cdot m)</math></b> | Depends on algorithm used internally                  |
| Reverse               | <code>s[::-1]</code>                | <code>reverse(s.begin(),s.end())</code> | <b><math>O(n)</math></b>                                | <b><math>O(n)</math></b>         | <b><math>O(n)</math></b>           | <b><math>O(n)</math></b>         |                                                       |
| Split / strip         | <code>s.split() / s.strip()</code>  | no built-in (manual loop)               | <b><math>O(n)</math></b>                                | <b><math>O(n)</math></b>         | <b><math>O(n)</math></b>           | <b><math>O(n)</math></b>         | C++ needs <code>stringstream</code> or manual parsing |

9. Quick-Glance Summary Table

| Need to...                                | Python structure                    | C++ structure               | Typical complexity            |
|-------------------------------------------|-------------------------------------|-----------------------------|-------------------------------|
| Fast index access                         | <code>list</code>                   | <code>vector</code>         | <b><math>O(1)</math></b>      |
| Fast insert/delete at both ends           | <code>deque</code>                  | <code>deque</code>          | <b><math>O(1)</math></b>      |
| Fast key→value lookup, no ordering needed | <code>dict</code>                   | <code>unordered_map</code>  | <b><math>O(1)</math> avg</b>  |
| Fast membership test, no ordering needed  | <code>set</code>                    | <code>unordered_set</code>  | <b><math>O(1)</math> avg</b>  |
| Key→value lookup, sorted order needed     | <code>SortedDict</code> (3rd party) | <code>map</code>            | <b><math>O(\log n)</math></b> |
| Membership test, sorted order needed      | <code>SortedSet</code> (3rd party)  | <code>set</code>            | <b><math>O(\log n)</math></b> |
| Repeated min/max extraction               | <code>heapq</code>                  | <code>priority_queue</code> | <b><math>O(\log n)</math></b> |
| LIFO stack behavior                       | <code>list</code>                   | <code>stack</code>          | <b><math>O(1)</math></b>      |
| FIFO queue behavior                       | <code>deque</code>                  | <code>queue</code>          | <b><math>O(1)</math></b>      |

**Naming cheat:** in C++, "unordered\_" prefix = hash table (fast, unsorted). No prefix (`map/set`) = tree (sorted, slower). In Python, `dict/set` are always hash tables — there's no naming distinction because there's no built-in sorted alternative.